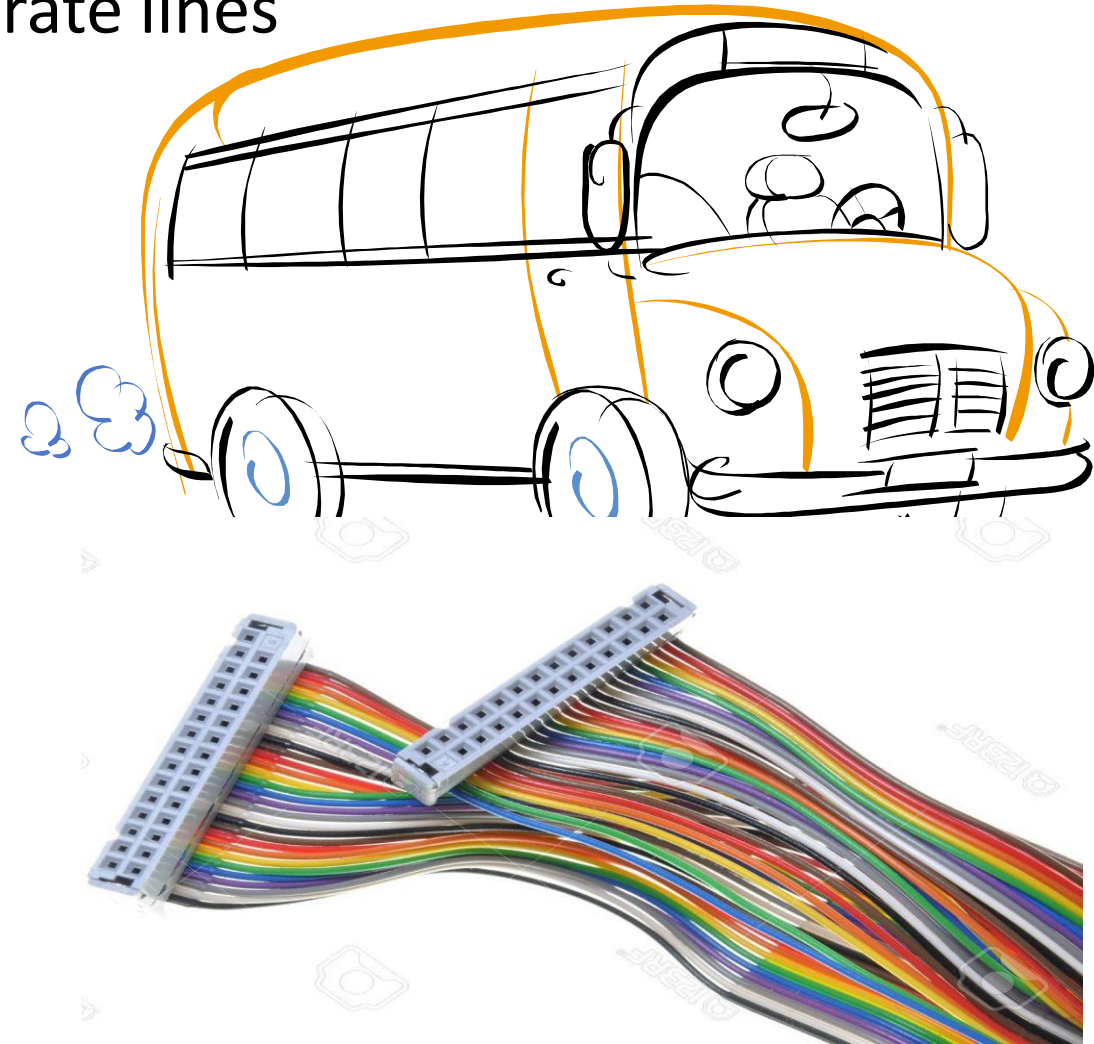


DATA BUS

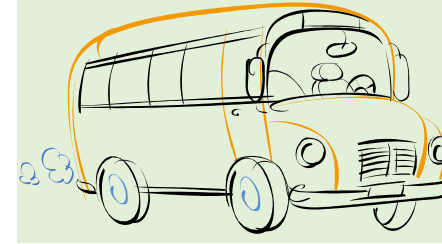
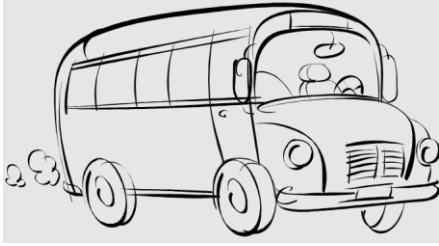
- Data lines that provide a path for moving data among system modules
- May consist of 16, 32, 64, 128, or more separate lines
- The number of lines is referred to as the *width* of the data bus.
- The number of lines determines how many bits can be transferred at a time.
- The width of the data bus is a key factor in determining overall system performance.



Source: 123rf.com

ADDRESS BUS

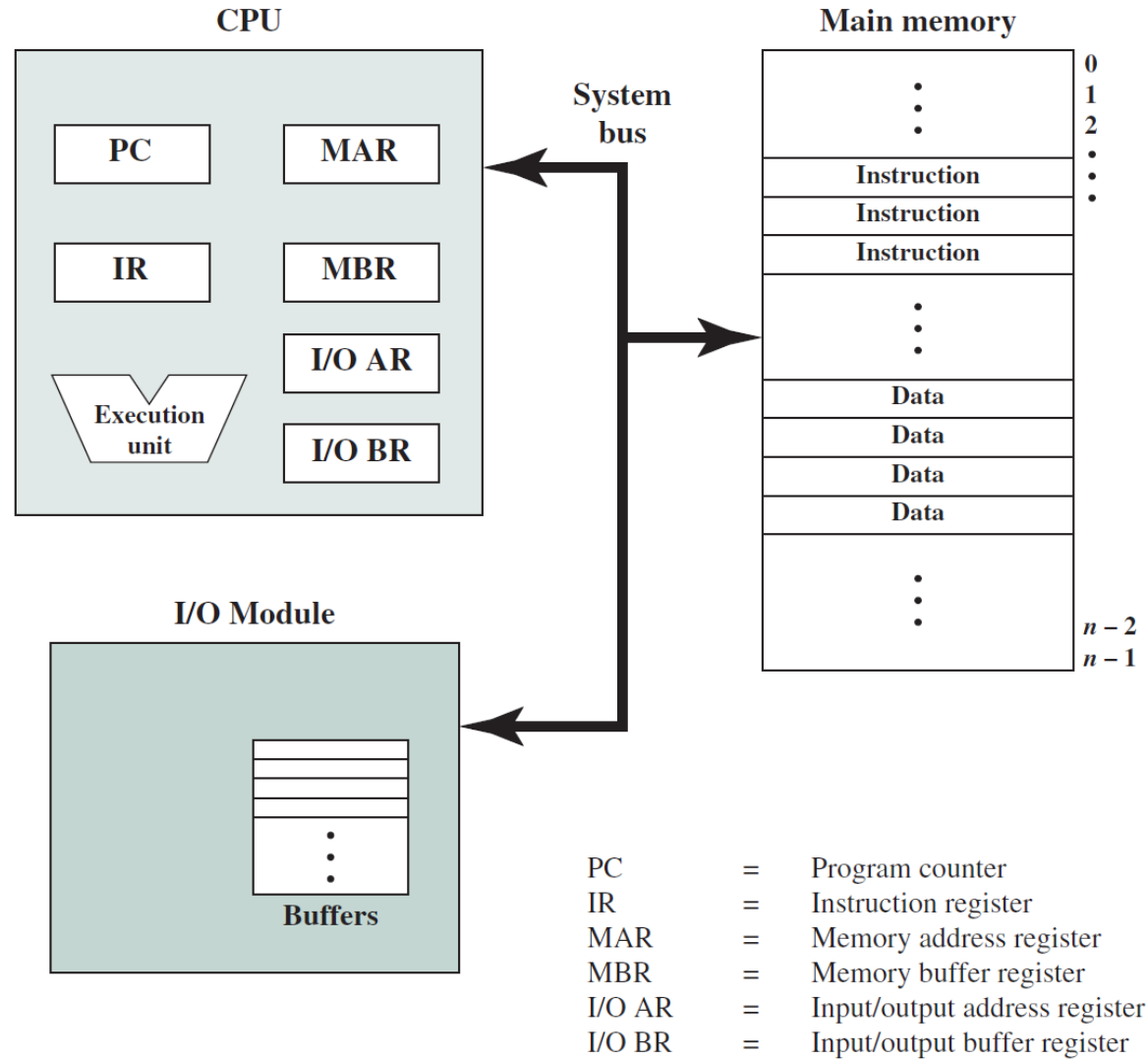
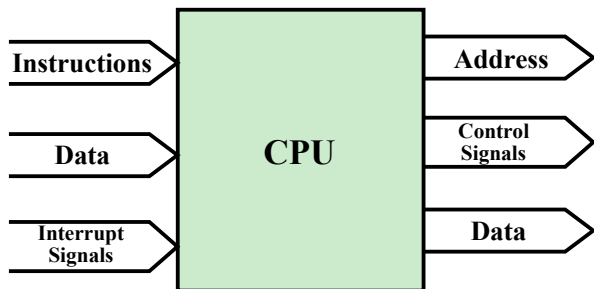
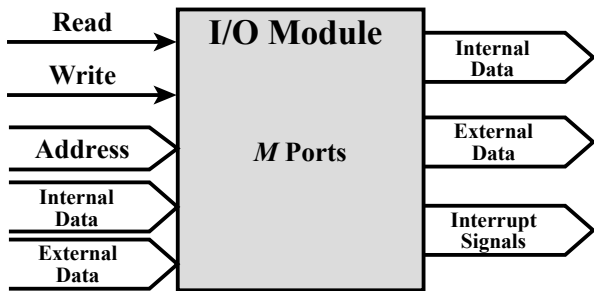
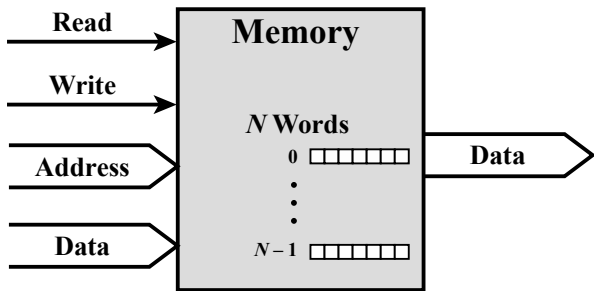
CONTROL BUS



- Used to designate the source or destination of the data on the data bus
 - If the processor wishes to read a word of data from memory it puts the address of the desired word on the address lines
- Width determines the maximum possible memory capacity of the system
- Also used to address I/O ports
 - The higher order bits are used to select a particular module on the bus and the lower order bits select a memory location or I/O port within the module

- Used to control the access and the use of the data and address lines
- Because the data and address lines are shared by all components there must be a means of controlling their use
- Control signals transmit both command and timing information among system modules
- Timing signals indicate the validity of data and address information
- Command signals specify operations to be performed

PEELING ANOTHER LAYER – CONCEPTUAL VIEW OF COMPUTER COMPONENTS

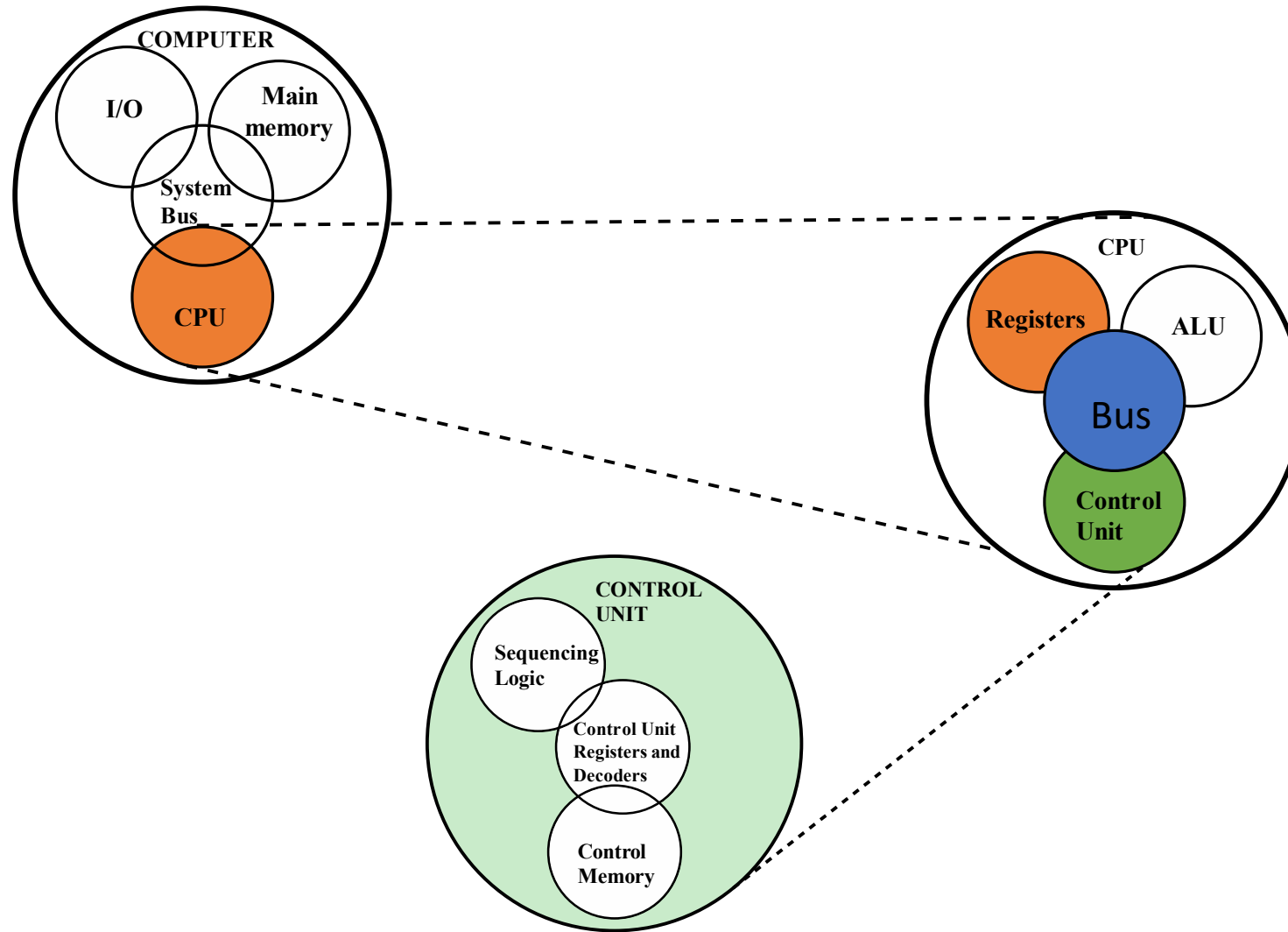


sum.c

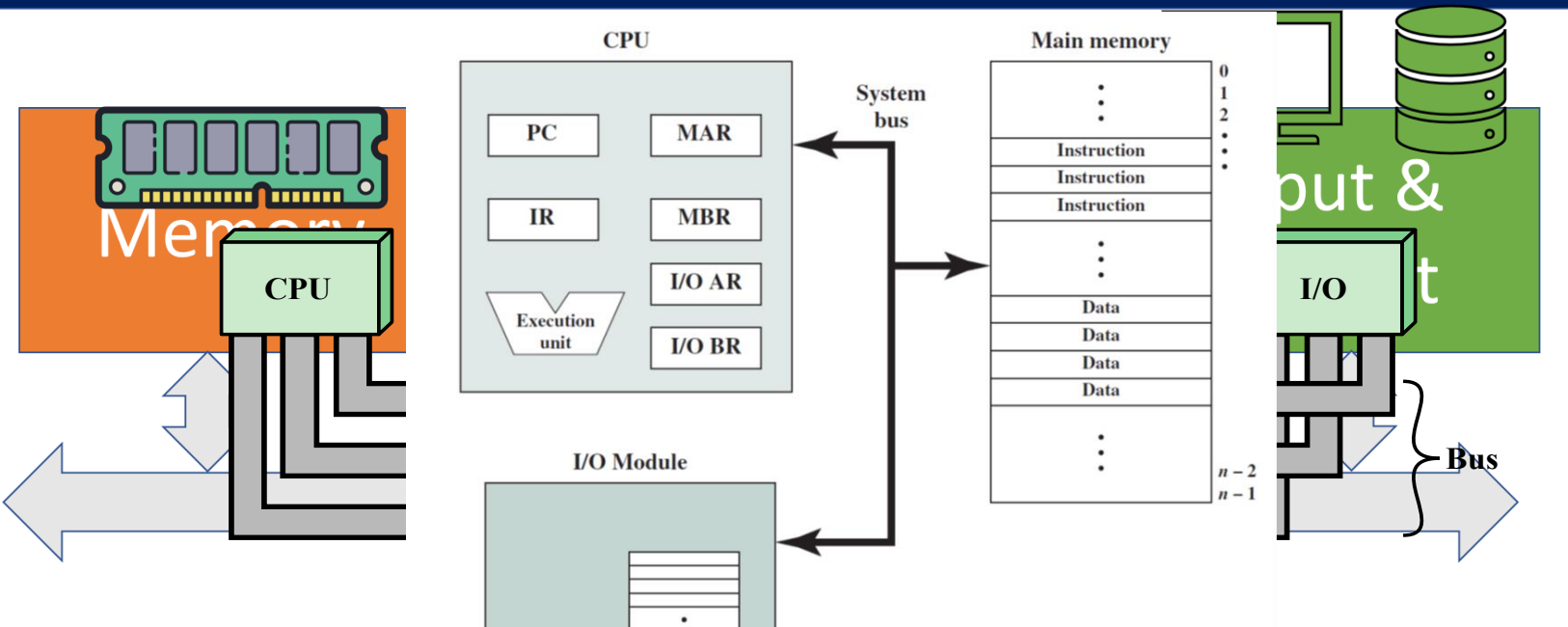
```

I1: a=b+c;
I2: x=d-e;
    
```

TOP-DOWN VIEW OF A COMPUTER – COMPUTER ORGANIZATION

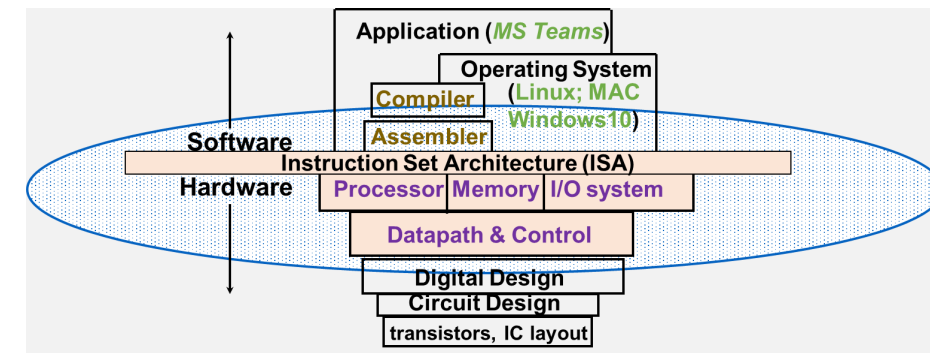


COMPUTER SYSTEMS – A CONCEPTUAL AND PRACTICAL VIEW



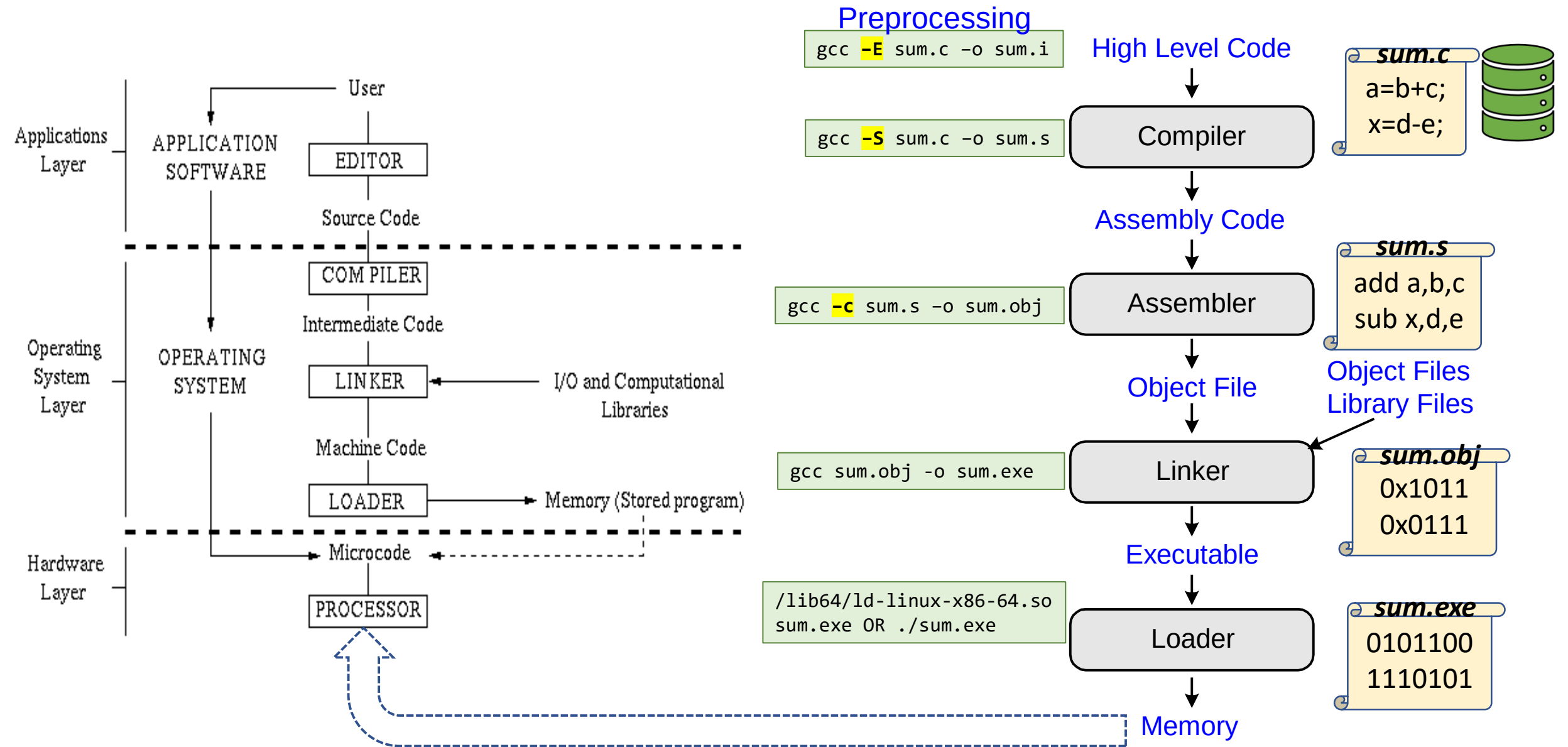
- Recollect:
 - Program needs to be eventually run on hardware
 - ISA is the abstraction to what hardware can do.
- Fundamental
 - Understand the basic principles involved in design of ISA.

PC = Program counter
IR = Instruction register
MAR = Memory address register
MBR = Memory buffer register
I/O AR = Input/output address register
I/O BR = Input/output buffer register



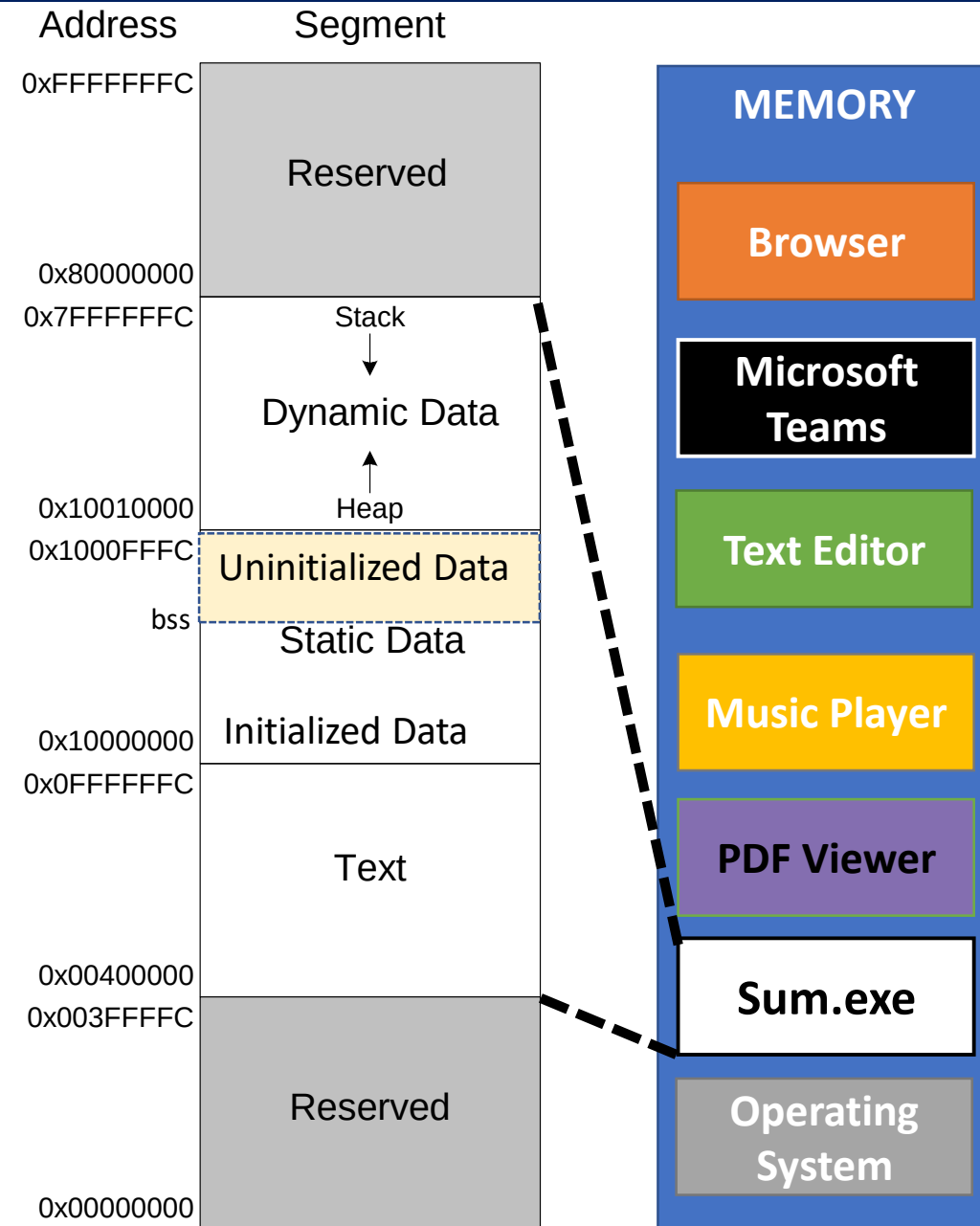
Chapter 2.12 (COAHP)

HW-SW INTERFACE



MEMORY MAP/LAYOUT OF A PROGRAM

- Instructions (also called *text or code*)
 - Set of Instructions to be executed.
- Data
 - **Global/Static**: allocated before program begins
 - **Dynamic**: allocated within the program
 - **Stack**: Intermediate/temporary store
- Everything is stored in Memory
 - How big is the memory?
 - What is the address range?
 - How to access data in this memory?



EXAMPLE PROGRAM: C CODE TO ASSEMBLY MAP

```
int x, y, z; // global variables
int sum(int a, int b) {
    return (a + b);
}

int main(void) {
    x = 2;
    y = 3;
    z = sum(x, y);
    return z;
}
```

Symbol	Address
X	0x10000000
Y	0x10000004
Z	0x10000008
Main	0x00400000
sum	0x0040002C

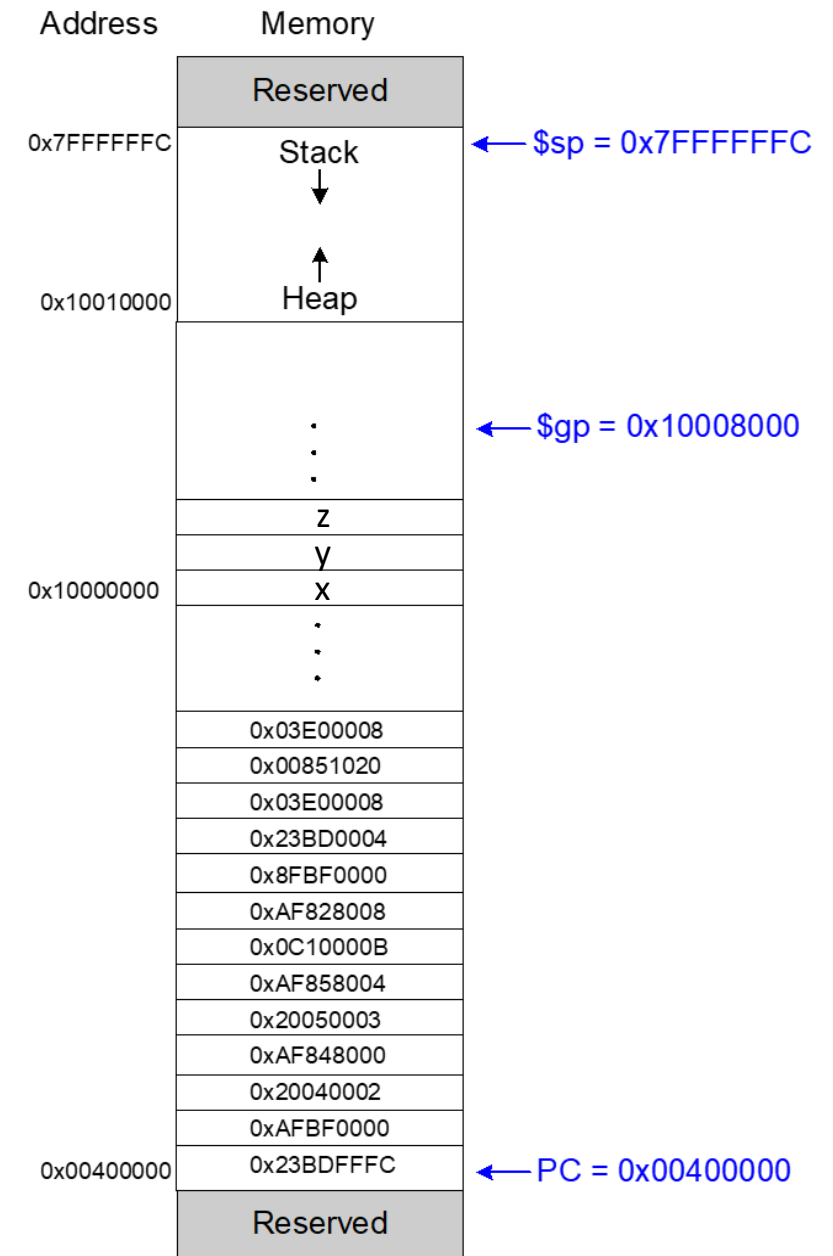
```
.data
x:
y:
z:
.text
main:
    addi $sp, $sp, -4    # stack frame
    sw   $ra, 0($sp)    # store $ra
    addi $a0, $0, 2      # $a0 = 2
    sw   $a0, x          # x = 2
    addi $a1, $0, 3      # $a1 = 3
    sw   $a1, y          # y = 3
    jal  sum             # call sum
    sw   $v0, z          # z = sum()
    lw   $ra, 0($sp)     # restore $ra
    addi $sp, $sp, 4     # restore $sp
    jr   $ra             # return to OS
sum:
    add  $v0, $a0, $a1   # $v0 = a + b
    jr   $ra             # return
```


EXAMPLE PROGRAM: EXECUTABLE

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	Address	Instruction
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	Address	Data
	0x10000000	x
	0x10000004	y
	0x10000008	z

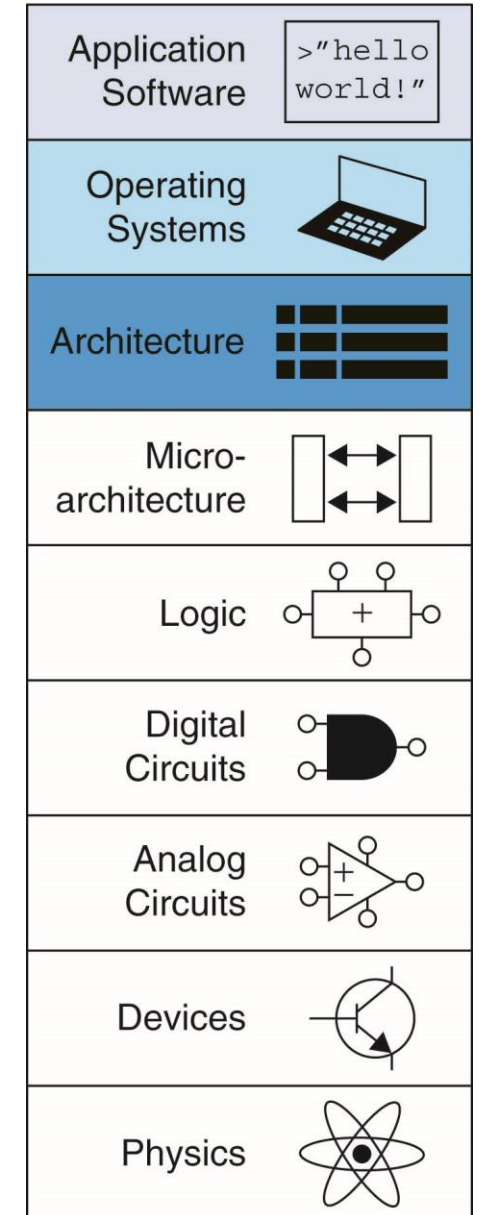
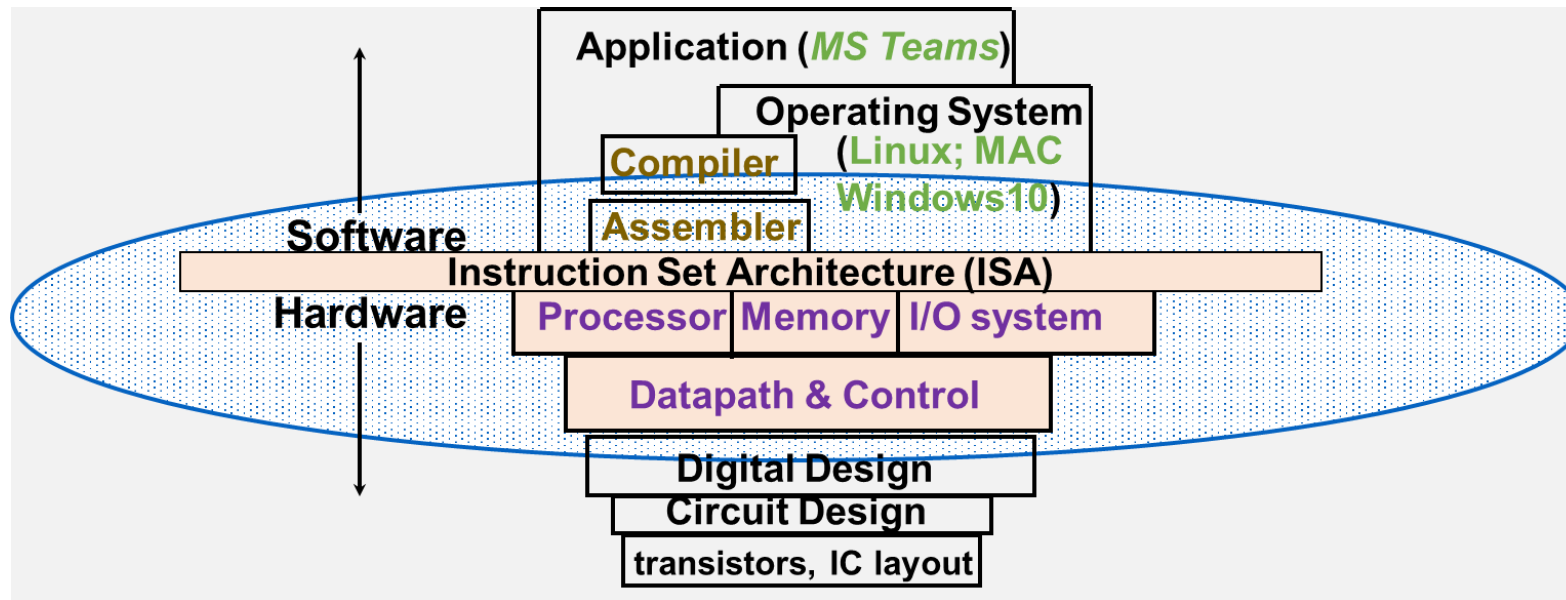
```

addi $sp, $sp, -4
sw  $ra, 0($sp)
addi $a0, $0, 2
sw  $a0, 0x8000($gp)
addi $a1, $0, 3
sw  $a1, 0x8004($gp)
jal  0x0040002C
sw  $v0, 0x8008($gp)
lw  $ra, 0($sp)
addi $sp, $sp, -4
jr  $ra
add $v0, $a0, $a1
jr  $ra
    
```



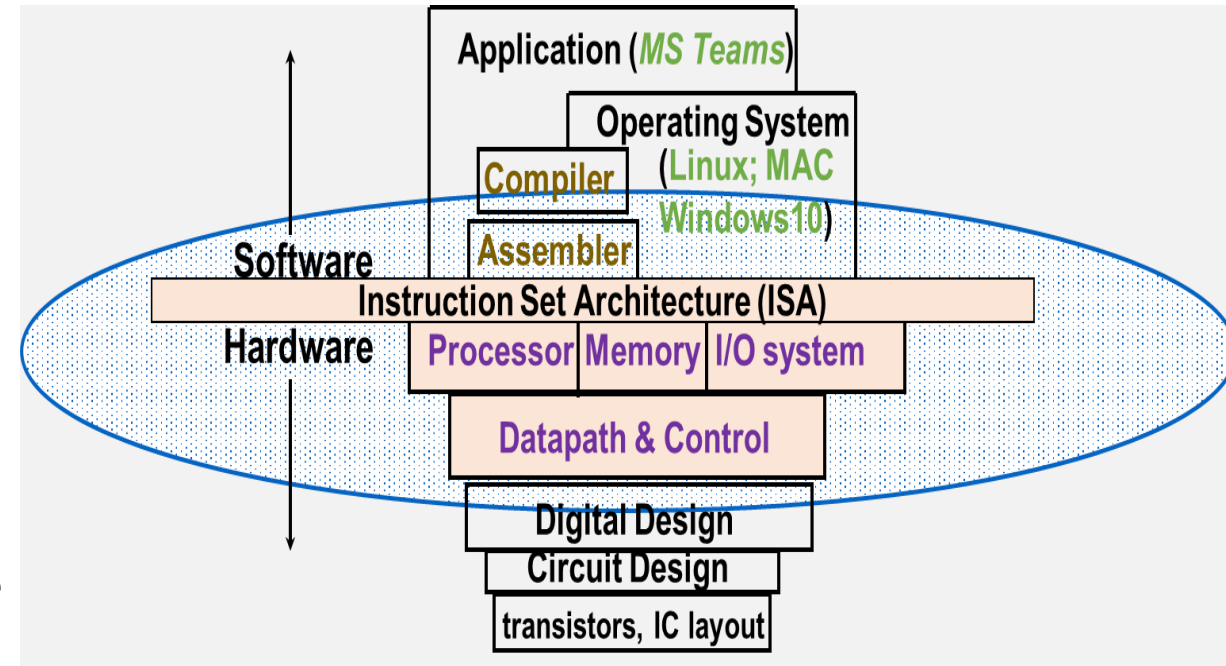
DEFINING ISA

- What is instruction set architecture (ISA)?
 - An abstract model, A programmers view of a computer.
 - A '**Contract**' or a well-defined Hardware Software Interface.
 - Every capability of a computer need to be expressible via ISA.



IMPORTANCE OF ISA

- The instruction set architecture (or ISA) is one of the most important design issues that a CPU designer must get right from the start.
- Features like caches, pipelining, superscalar implementation, etc., can all be grafted on to a CPU design long after the original design is obsolete.
- It is very difficult to change the instructions a CPU executes once the CPU is in production and people are writing software that uses those instructions.
- Therefore, one must carefully choose the instructions for a CPU.



DESIGNING ISA: INSTRUCTION SET DESIGN

Defines many of the functions performed by the processor

Programmer's means of controlling the processor

Very complex because it affects so many aspects of the computer system

ISA: Fundamental design issues

Operation repertoire

- How many and which operations? How complex should they be?

Data types

- Various types of data upon which operations are performed

Instruction format

- Instruction length in bits, number of addresses, size of various fields, etc.

Registers

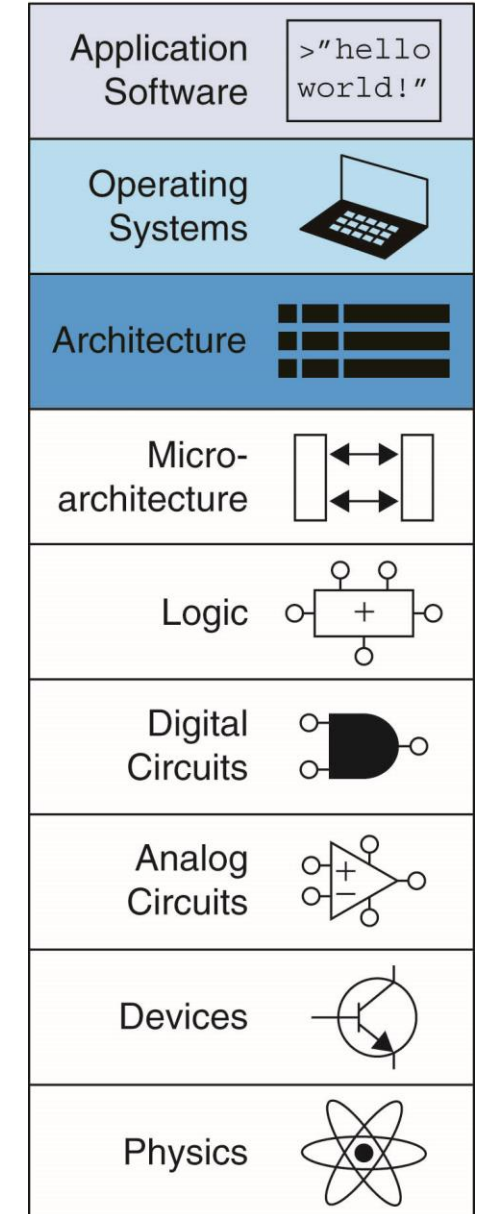
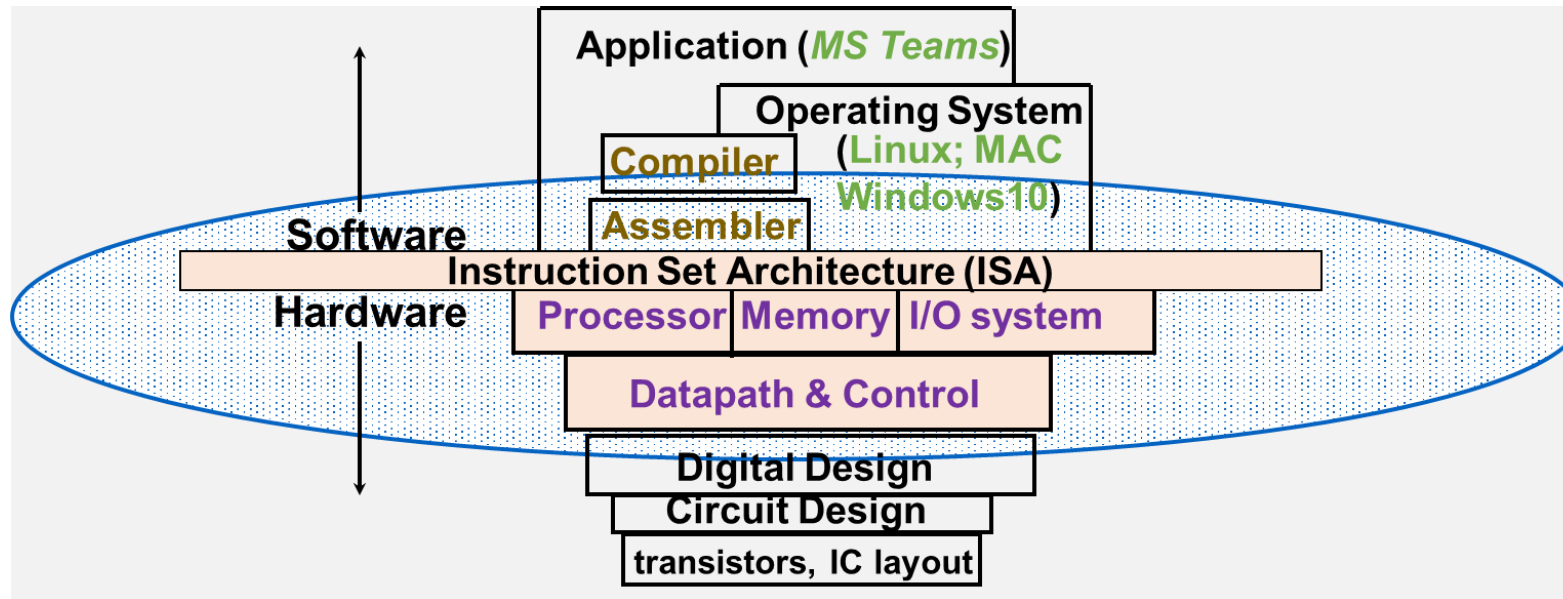
- Number of processor registers that instructions can use

Addressing

- The mode or modes by which the address of an operand is specified

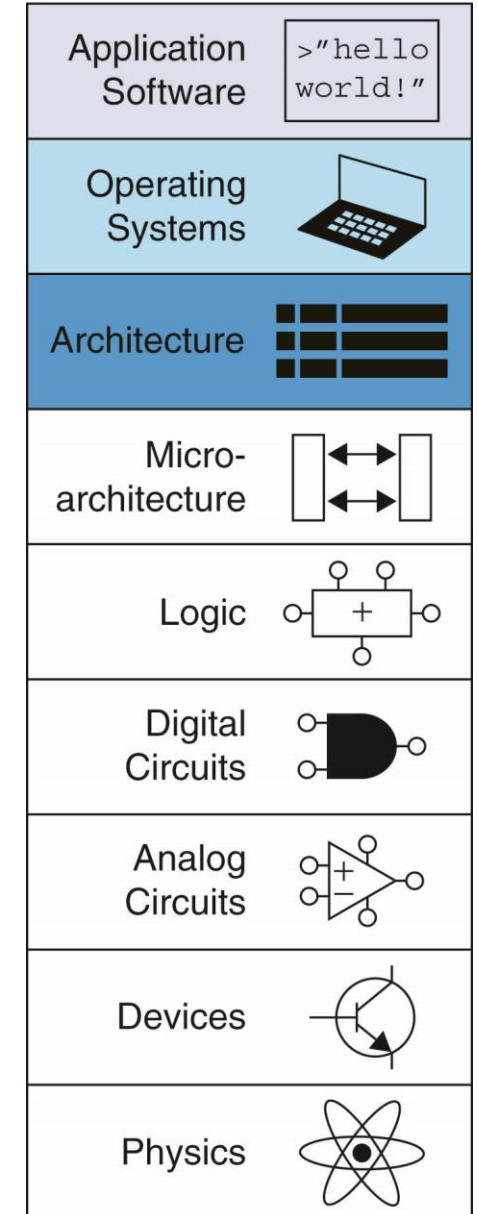
RE-DEFINING ISA

- What is not part of instruction set architecture (ISA)?
 - How operations are implemented
 - Which operations are fast and which are slow and when
 - Which operations take more power and which take less



(IN) FORMALLY CHARACTERIZING THE ISA

- ISA – Instructions from the Software to the Hardware.
 - Defines **Registers**.
 - Defines **Data types**.
 - Defines **Data Operations**.
 - Defines **Memory Size**
- Defines **data transfer modes** (instructions) between registers, memory and I/O
- There should be *sufficient* instructions to efficiently translate any program for machine processing.
- Instructions be easily translatable to the binary representation used by the hardware.
- Instructions can be of variable-length or fixed-length.



A LANGUAGE ANALOGY FOR ISAs

- Communication
 - Person-to-person → software-to-hardware
- Similar structure
 - Narrative → program
 - Sentence → **instruction**
 - **Verb → operation (add, multiply, load, branch)**
 - **Noun → data item/operand (immediate, register value, memory value)**
 - **Adjective → addressing mode**
- Many different languages, many different ISAs
 - Similar basic structure, details differ (sometimes greatly)
- Key differences between languages and ISAs
 - Languages evolve organically, many ambiguities, inconsistencies
 - **ISAs are explicitly engineered and extended, unambiguous**

PROGRAM TO ISA MAP

```
#define ENABLE_PROFILING (1)
#ifdef ENABLE_PROFILING
#include "timer_util.h"
#endif

unsigned long int find_largest_num (unsigned long int *arr, unsigned long int arr_size) {
    // Finding the largest number
    unsigned long int largest_num = 0;
    unsigned long int i = 0;
    for (i; i < arr_size; ++i) {
        if (largest_num < arr[i]) {
            largest_num = arr[i];
        }
    }
    return largest_num;
}

unsigned long int find_sum (unsigned long int *arr, unsigned long int arr_size) {
    // Finding the largest number
    unsigned long int total_num = 0;
    unsigned long int i = 0;
    for (i; i < arr_size; ++i) {
        total_num += arr[i];
    }
    return total_num;
}
```


PROGRAM TO ISA MAP

```
int main() {
    unsigned long int num, largest_num, *arr;
    printf("Enter the total number of elements: ");
    scanf("%ld", &num);
    // Allocating memory for num elements
    arr = (unsigned long int *)malloc(sizeof(unsigned long int)*num);
    if (arr == NULL) {
        printf("Error!!! memory not allocated.");
        exit(0);
    }
    // Storing numbers entered by the user.
    for (unsigned long int i = 0; i < num; ++i) {
        printf("Enter Number %ld: ", i + 1);
        scanf("%ld", arr + i);
    }
#ifdef ENABLE_PROFILING
    unit64_t start_time = timer_util_get_current_cpu_cycles();
#endif
    largest_num = find_largest_num(arr, num);
#ifdef ENABLE_PROFILING
    unit64_t elapsed_cycles = timer_util_get_elapsed_cpu_cycles(start_time);
#endif
    printf("Largest number = %ld", largest_num);
    return 0;
}
```

FROM PROGRAM EXECUTION TO ISA MAPPING

```
int main() {
    unsigned long int num, largest_num, *arr;
    printf("Enter the total number of elements: ");
    scanf("%ld", &num);
    // Allocating memory for num elements
    arr = (unsigned long int *)malloc(sizeof(unsigned long int)*num);
    if (arr == NULL) {
        printf("Error!!! memory not allocated.");
        exit(0);
    }
    // Storing numbers entered by the user.
    for (unsigned long int i = 0; i < num; ++i) {
        printf("Enter Number %ld: ", i + 1);
        scanf("%ld", arr + i);
    }
#ifdef ENABLE_PROFILING
    unit64_t start_time = onvm_util_get_current_cpu_cycles();
#endif

    largest_num = find_largest_num(arr, num);
#ifdef ENABLE_PROFILING
    unit64_t elapsed_cycles = onvm_util_get_elapsed_cpu_cycles(start_time);
#endif
    printf("Largest number = %ld", largest_num);
    return 0;
}
```

```
#define ENABLE_PROFILING (1)
#ifdef ENABLE_PROFILING
#include <timer_util.h>
#endif
unsigned long int find_largest_num (unsigned long int *arr, unsigned long int arr_size) {
    // Finding the largest number
    unsigned long int largest_num = 0;
    unsigned long int i = 0;
    for (i; i < arr_size; ++i) {
        if (largest_num < arr[i]) {
            largest_num = arr[i];
        }
    }
    return largest_num;
}
```

```
unsigned long int find_sum (unsigned long int *arr, unsigned long int arr_size) {
    // Finding the largest number
    unsigned long int total_num = 0;
    unsigned long int i = 0;
    for (i; i < arr_size; ++i) {
        total_num += arr[i];
    }
    return total_num;
}
```

TYPES OF INSTRUCTIONS NECESSARY TO ACCOMPLISH A PROGRAM

```
Sum = Num1 + Num2
Dif = Num1 - Num2
Pro = Num1 * Num2
Quo = Num1 / Num2
Rem = Num1 % Num2
```

Data Manipulation (Operations on Data)

- Arithmetic Operations
- Logical Operations
- Bitwise Operations
- Relational Operations

```
A = 100
B = A*10
C = A
Str = "HelloWorld"
*ptr = &str[5]
Memcpy(a,b,10)
```

Data Access/Transfer Operations:

- *Load,*
- *Store,*
- *Move*
- *Assignment Operations!*

```
While I > 100;
For(;i<100;i++)
Switch(stud_id)
If(a > 10) {};
Else if (a < 20){}
Else {}
```

Control Transfer (Decision Making)

- UnConditional: (Procedure Call)
- Conditional:(Branch/Loop)

```
System calls:
    sbrk,
    exit, exit2
Assembler
directives : .file
            .align .section
```

Special Instructions

- Status Manipulation
- Debug support
- NOP

Instruction Set Architecture

SOME KEY ATTRIBUTES OF ISAs

- Instruction encoding
 - Fixed length (32-bit or 64-bit for MIPS, ARM)
 - Variable length (1 byte to 16 bytes, average of ~3 bytes, x86_64)
- Number and type of registers
 - MIPS has 32 “integer” registers and 32 “floating point” registers
 - ARM & x86 both have 16 “integer” registers and 16 “floating point” registers
- Address space
 - ARM: 32-bit addresses at 8-bit granularly (2^{32} or 4GB total)
 - Modern x86 and ARM64: 64-bit addresses (2^{32} or 16 ExaB!)
- Memory addressing modes
 - MIPS: address calculated by “register+offset”
 - x86 and others have much more complicated addressing modes.

TYPES/FAMILY OF ISA

- Complex instruction set computing (CISC)
 - Many instructions (several hundreds)
 - An instruction takes many cycles to execute
 - Examples: _x86(Intel Pentium), VAX (vax11), PDP(pdp8)
- Reduced instruction set computing (RISC)
 - Small set of instructions (typically 32)
 - Simple instructions, each executes in one clock cycle – ***REALLY? Well, almost.***
 - Effective use of pipelining
 - Examples: ARM (snapdragon), MIPS (MIPS64), RISC-V (Raven-1)
- There are several others too:
 - VLIW (Very Large Instruction Word)
 - Minimal Instruction Set Computers (MISC)
 - Zero Instruction Set Computers (ZISC) etc.